

프론트엔드 개발



1. 화면 스타일 설정

❖화면 캡처한 후 스타일 가이드 생성

■ 화면 캡처

N 캘린더

일정 쓰기

기념일 관리

2025.10

1년

월

월

화

수

목

금

토

28

29

30

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

1

31

전체 일정 보기

내 캘린더

[기본] 내 캘린더

내 할 일

구독 캘린더

구글 캘린더

★ 중요 일정 보기

📅 범주 일정 보기

👤 캘린더 일정 정리하기

📄 삭제된 일정 목록

오늘 할 일 0건

일정 검색

상세

내 캘린더

2025.10

오늘

일간

주간

월간

목록

평일

일

월

화

수

목

금

토

28

(음) 08.07

29

30

1

2

3

개천절

4

5

추석 연휴

6

추석

7

추석 연휴

8

대처공휴일(추석)

9

한글날

10

11

12

(음) 08.21

13

14

15

16

17

18

19

20

21

(음) 09.01

22

23

상강

24

25

26

27

(음) 09.07

28

29

30

31

1

오늘 할 일 0건

2

1. 화면 스타일 설정

- 캡처한 이미지를 Claude Code에 Alt+V로 첨부한 후 프롬프트 작성
 - "이 화면 이미지 정보를 참조하여 프론트엔드 앱에 적용할 수 있는 스타일 가이드를 작성해줘"
 - "이 화면 이미지 정보를 참조하여 tailwind를 적용할 때의 스타일 가이드를 생성하여 @docs\w 디렉토리에 APP_STYLE_GUIDE.md 파일로 저장해줘"
- 생성된 스타일 가이드 예시

```
# 프론트엔드 앱 스타일 가이드
Team CalTalk 프론트엔드 앱에 적용된 네이버 캘린더 스타일
시스템 가이드입니다.

### 1. Tailwind Config (`tailwind.config.js`)
네이버 캘린더 색상 팔레트와 커스텀 설정이 추가되었습니다.

#### 색상 시스템
```js
naver: {
 green: '#10b981', // 로고, 오늘 날짜
 blue: '#3b82f6', // 토요일, 강조 버튼
 red: '#ef4444', // 일요일, 공휴일
 purple: '#7c3aed', // 특별 일정
 gray: {...} // 배경, 테두리
}
```
```

```
### 2. 전역 CSS (`src/index.css`)

#### CSS 변수
```css
--naver-green: 16 185 129;
--naver-blue: 59 130 246;
--naver-red: 239 68 68;
--primary: var(--naver-green);
--accent: var(--naver-blue);
```

#### 컴포넌트 클래스
- `.naver-header`: 헤더 스타일
- `.naver-logo`: 로고 스타일
- `.naver-sidebar`: 사이드바 스타일
- `.naver-create-button`: 생성 버튼
- `.naver-calendar-cell`: 캘린더 셀
- `.naver-today-badge`: 오늘 날짜 배지
```

2. 개발전 사전 작업

❖프론트엔드 개발을 위한 사용자 정의 Command 또는 Skill 준비

▪ ./claude/skills/issue-resolver-frontend/SKILL.md

```
---
name: issue-resolver-frontend
description: 깃헙 이슈 번호를 입력받아 프론트엔드 개발을 수행하는 사용자 정의 Skill
arguments: ISSUE_NUMBER
---
```

너는 Github Issue를 해결하는 유능한 프론트엔드 개발자이다. \${ISSUE_NUMBER} 를 Argument로 전달받아 해당 번호의 Issue를 프론트엔드 문제를 해결한다.

작업 내용

- 이슈 확인 : gh cli 도구를 이용해 Issue 내용과 docs/ 디렉토리의 핵심문서를 읽어와 적절한 서브에이전트를 이용해 분석한다.
- 자식 브랜치 분기 : feature-\${ISSUE_NUMBER} 형태의 자식 브랜치를 생성한다.
- 기존 코드 분석 : 적절한 서브에이전트를 사용해 이슈정보, docs 디렉토리의 문서들, 코드베이스(frontend 디렉토리)의 코드를 분석한다.
- 이슈확인, 자식 브랜치 분기, 기존 문서 분석, 코드베이스 코드 분석은 병렬로 실행한다.
- 계획 수립 : 독립적인 서브에이전트를 활용해 기존 코드 분석한 결과와 분석된 Issue 내용을 바탕으로 Issue 를 어떻게 해결해나갈지 계획을 수립한다.
- 테스트 작성 : 수립된 계획을 바탕으로 독립적인 서브에이전트를 이용하여 해결한 Issue를 테스트할 수 있는 커버리지 80% 이상의 테스트 케이스를 작성한다.
- 문제 해결 : 수립된 계획을 바탕으로 프론트엔드 개발에 적합한 서브에이전트를 선택해서 해결한다.
- 테스트 작성과 문제 해결은 병렬로 수행한다.
- 테스트 수행 : 독립적인 서브에이전트를 이용해 미리 작성한 테스트를 이용해 테스트한다.
- 테스트가 완결되면 feature-\${ISSUE_NUMBER} 브랜치를 리포지토리에 커밋, 푸시하고 PR을 생성한다.

2. 개발전 사전 작업

❖깃헙 이슈 생성

- 필수는 아니지만 장점이 많음.
 - 효율적인 협업 : 이슈에 댓글을 다는 등의 효과적인 소통이 가능함
 - 조직화 : 레이블을 이용해 이슈를 분류하고 마일스톤을 이용해 버전, 스프린트 별로 묶어서 관리할 수 있음
 - 투명한 진행상황 : 모든 개발 진행과정이 기록됨
- 백엔드 개발 시에 설정 완료.
- 백엔드 개발 시의 가이드 참조

전문화된 서브에이전트를 이용해 @docsW8-plan.md 의 처리 단계를 유지하면서 @docsW 디렉토리의 문서들을 참조하여 깃헙 이슈를 생성해줘.

반드시 지켜야할 것

- Title : Stage를 명시할 것
- Label : 종류, 영역, 복잡도
- Todo : 해야할 일 정보
- 완료 조건
- 기술적 고려사항
- 의존성
- 선행, 후행 작업

2. 개발전 사전 작업

❖하위 CLAUDE.md 지침

- 프론트엔드 개발을 진행하고 있는 하위 디렉토리(예: frontend)에 CLAUDE.md 지침을 정의함
- 반드시 포함할 내용
 - "오버엔지니어링 금지"
 - "지침에 있는 기능만을 정확하게 구현할 것"
- AI 모델이 창의적으로 지침 이외의 작업을 수행하지 않도록 강제할 것.
- 하위 지침을 만들 때 AI 도구를 활용할 수 있음
 - 생성된 지침을 검토하고 간결하게 유지

2. 개발전 사전 작업

❖프론트엔드 개발을 위한 mcp 설정 추가

```
{
  "mcpServers": {
    "shadcn": {
      "command": "npx", "args": [ "-y", "shadcn@latest", "mcp" ]
    },
    "context7": {
      "command": "npx", "args": [ "-y", "@upstash/context7-mcp" ]
    },
    "playwright": {
      "command": "npx", "args": [ "-y", "@playwright/mcp@latest" ]
    },
    "chrome-devtools": {
      "command": "npx", "args": [ "-y", "chrome-devtools-mcp@latest" ]
    }
  }
}
```

3. 프론트엔드 개발

❖ 사용자 정의 Skill을 이용해 빠르게 개발

- 앞에서 만들어둔 issue-resolver-frontend 를 이용해 빠르게 개발
 - /issue-resolver-frontend 이슈번호 추가적인 지침
- 사용자 정의 Command에 의해 개발, 테스트 완료
- 각 단계를 거치면서 자동화된 테스트를 수행하겠지만 오류가 발생함.
 - AI가 설계 문서대로 개발하지 않고, 창의성을 발휘하는 경우
 - AI를 이용해 설계한 문서가 완벽하지 않은 경우

❖ 테스트 완료 후

- 변경/추가 사항을 feature 브랜치에 스테이징-커밋-푸시
- main 브랜치로의 PR(Pull Request) 생성
- PR 사항을 기반으로 코드 리뷰 후 승인 / 병합
- 해결된 이슈는 Close함

3. 프론트엔드 개발

❖ 주의사항

- 정확한 기능의 테스트를 위해 백엔드와 프론트엔드를 모두 실행한 후 테스트함.
- 백엔드 API 엔드포인트는 반드시 환경변수를 이용하도록 코딩할 것
- 프론트엔드 프로젝트 기획시 기술 스택을 선정했다면 반드시 그 기술 스택, 아키텍처를 사용하도록 지침을 지정할 것
 - 예시 : 백엔드 API 호출 기술 스택 : axios? tanstack query? fetch?
 - 기술 스택을 프론트엔드 프로젝트 디렉토리의 하위 CLAUDE.md에 명시할 것
 - 프론트엔드 디렉토리 구조를 미리 설계했다면 해당 문서를 지정하여 프롬프트가 실행되도록 할 것
- 브라우저에서 초기 로그인, 사용자 생성, 샘플 데이터 생성 등의 작업이 필요할 수 있음
- 프론트엔드 개발 도중 백엔드의 설계 사항이 변경될 수 있는데, 이 경우 반드시 관련 문서를 업데이트해야 함
 - 예시 1) 백엔드 API의 엔드포인트의 내용이 변경되었다면 swagger.json을 반드시 업데이트함.
 - 예시 2) 데이터베이스 스키마가 변경되는 경우 ERD와 DB 설계문서를 업데이트함
- 자동화된 테스트에만 의존하지 말고 수동 테스트도 병행할 것
 - AI가 생성한 테스트 코드, 시나리오가 완벽하지 않을 수 있음
 - 수동 테스트를 통해 확인한 내용을 테스트 시나리오에 다시 반영할 것

3. 프론트엔드 개발

❖디버깅

- 오류 발생시 오류 메시지를 프롬프트로 제공하는 것이 기본적인 디버깅 방법

❖로깅의 중요성

- AI가 모든 것을 해결해주려면 대단히 많은 토큰과 시간을 필요로 함
 - 원인을 파악하기 위해서 많은 토큰 소모, 반복적인 분석 작업을 하게 됨.
- 따라서 충실하게 로깅하도록 개발하고, 로그를 기반으로 분석하도록 함
 - 개발환경에서만 로깅, 프로덕션 환경에서는 로깅하지 않도록 logger 함수 작성할 것을 권장
- 가능하다면 단순한 에러메시지 뿐만 아니라 로그에 기록된 것을 직접 AI에게 알려주면 더 빠르게 디버깅할 수 있음

❖기술 이해의 중요성

- 개발자가 에러메시지와 관련한 기술 개념을 이해하고 해당 지점과 힌트를 AI에게 알려주면 더 좋은 결과를 얻을 수 있음
- chrome-devtools mcp를 활용한 테스트/디버깅 권장함
 - "chrome-devtools mcp를 사용해 네트워크 탭에서 실패한 요청들을 확인하고 상태 코드와 함께 정리해줘"
 - "chrome-devtools mcp를 사용해 폼 제출 시 유효성 검사가 제대로 작동하는지 확인해줘"

4. 견고한 애플리케이션 개발

❖ SOLID 원칙

- 유지보수가 용이하고 견고한 애플리케이션을 개발하기 위한 5가지 객체지향 설계 원칙
- 원칙
 - 원칙 1 : 단일 책임 원칙 (SRP: Single Responsibility Principle)
 - 클래스, 모듈에는 단 하나의 책임만 가지도록 설계
 - 한 클래스가 수행하는 기능이 여러 개라면 내부 기능(메서드)끼리 강하게 결합될 수 있음
 - 원칙 2 : 개방,폐쇄 원칙 (OCP: Open Closed Principle)
 - 확장에는 열려 있고, 수정에는 닫혀 있도록 설계.
 - 기존 코드를 수정하지 않고 새로운 기능을 추가할 수 있어야 함
 - 원칙 3 : 리스코프 치환 원칙 (LSP: Liskov Substitution Principle)
 - 자식 클래스는 부모 클래스를 대체할 수 있어야 함.
 - 상속 관계가 아닌 클래스들을 상속 관계로 설정 하면 이 원칙에 위배됨
 - 원칙 4 : 인터페이스 분리 원칙 (ISP: Interface Segregation Principle)
 - 특정 클라이언트는 자신이 사용하지 않는 인터페이스는 구현하지 않아야 함
 - 클라이언트가 사용하지 않는 인터페이스에 변경이 발생하더라도 해당 클라이언트는 영향을 받지 않음
 - 원칙 5 : 의존성 역전 원칙 (DIP: Dependency Inversion Principle)
 - 상위 모듈을 하위 모듈에 의존해서는 안됨. 추상 클래스나 인터페이스에 의존해야 함
 - 하위 모듈이 변경되더라도 상위 모듈은 변경이 불필요하도록 설계

4. 견고한 애플리케이션 개발

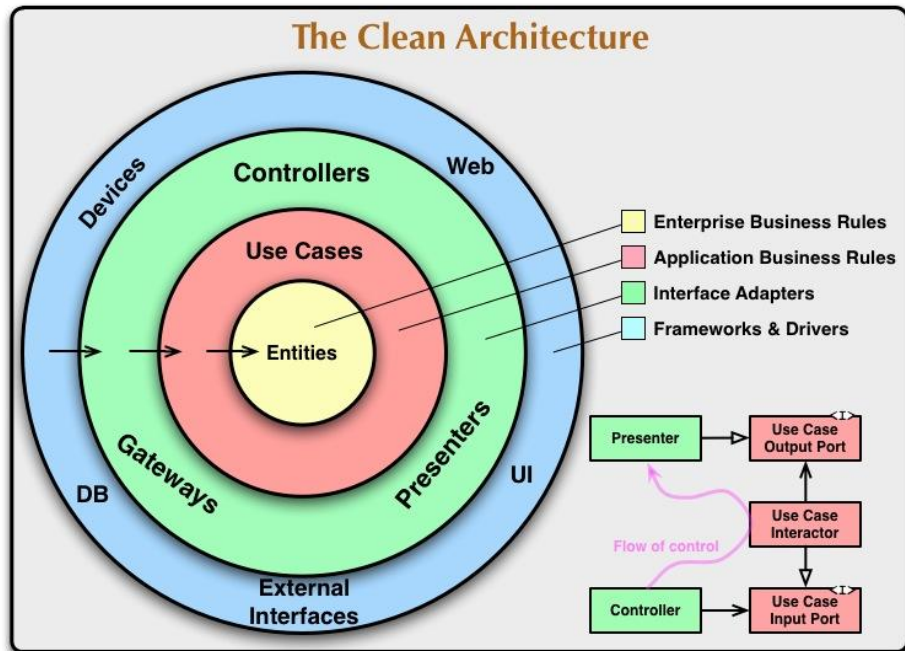
❖ Clean Architecture

■ 개념

- Robert C. Martin(Uncle Bob)이 제안한 소프트웨어 아키텍처 패턴
- 관심사의 분리를 통해 유지보수가 쉽고 테스트가 용이한 시스템 개발을 목표로 함

■ Clean Architecture의 계층

- 동심원으로 이루어진 4개의 계층



4. 견고한 애플리케이션 개발

■ 계층 : 가장 안쪽부터 바깥쪽으로

- Entities : 시스템의 비즈니스 규칙을 캡슐화
- Use Cases : 애플리케이션의 비즈니스 로직을 수행함. Entity와 데이터 흐름을 조정
- Interface Adapters
 - 외부 계층(프레임워크, 데이터베이스, UI 등)에서 사용할 수 있는 형태로 데이터를 변환.
 - Controller, Presenter, Gateway 등이 이 계층에 속함
- Frameworks and Drivers
 - 데이터베이스, 프레임워크, UI 등 구체적인 구현
 - 가장 바깥쪽 계층

■ 핵심 원칙

- 관심사 분리
 - 애플리케이션의 각 부분을 독립적인 계층으로 분리하여 각 부분이 한 가지 역할을 전담하도록 함
- 의존성 규칙
 - 의존성은 바깥 계층에서 안쪽 계층으로 향해야 함
 - 예를 들어 Use Cases에 해당하는 비즈니스 로직은 데이터베이스, UI 등의 세부 사항에 의존해서는 안됨
- 프레임워크 독립성
 - 특정 프레임워크나 외부 라이브러리에 종속되지 않도록 설계해야 함
 - 프레임워크가 변경되더라도 핵심 비즈니스 로직은 영향을 받지 않아야 함

4. 견고한 애플리케이션 개발

❖ SOLID 원칙, Clean Architecture를 적용하는 방법

- 프롬프트로 간결한 내용을 프로젝트 수준 지침으로 지정
- AI는 이미 SOLID 원칙, Clean Architecture를 알고 있으므로 장황하게 프롬프트 할 필요 없음
- 예시
 - "SOLID 원칙 반드시 준수"
 - "Clean Architecture 반드시 적용"

5. 통합 테스트

❖ Playwright를 이용한 E2E 테스트

- Playwright MCP를 이용해 사용자 시나리오 단계별로 테스트 수행
 - 브라우저를 열어서 자동으로 테스트를 수행하고 스크린 스냅샷을 저장함.
 - 테스트 리포트 자동 작성
- 테스트 리포트를 확인하면서 추가적인 테스트, 권장 테스트를 추가 수행함.

❖ E2E 테스트 실패라면?

- 직접 브라우저를 열어서 수동 테스트도 수행해야 함.
- 통합테스트 시에 테스트 API 코드에 오류가 있을 수 있음
- 수동 테스트한 결과를 프롬프트로 알리고 테스트를 재시도하도록 해야 함

5. 통합 테스트

❖ 통합 테스트를 위한 프롬프트 예시

- 현재 백엔드와 프론트엔드가 모두 개발서버로 실행중이야. playwright-mcp를 이용해 @docs/3-user-scenario.md를 참조해 통합테스트를 수행해줘. 테스트 결과는 test/e2e 디렉토리에 저장해

❖ 통합 테스트 결과 예시 1

Team CalTalk E2E 통합 테스트 최종 리포트

테스트 날짜: 2025-10-05 테스트 환경: Development (localhost:5173 Frontend, localhost:3000 Backend) 테스트 도구: Playwright MCP
테스트 대상: 전체 사용자 시나리오 1~4 (docs/3-user-scenarios.md 기반) 테스트 상태:  전체 성공 (100% PASS)

전체 테스트 개요

Team CalTalk의 핵심 워크플로우 4개 시나리오를 완전히 검증하였습니다. 초대 코드 기반 팀 참여, 일정 관리, 충돌 감지, 변경 요청 워크플로우, 권한 시스템이 모두 정상 작동함을 확인했습니다.

| 테스트 진행 순서

1. 세션 1 (14:00~14:20): 시나리오 1, 2 완료 → 9개 스크린샷
 2. 세션 2 (14:25~14:32): 시나리오 3, 4 완료 → 8개 스크린샷
-

5. 통합 테스트

❖통합 테스트 결과 예시 2

✅ 테스트 결과 종합

| 시나리오 | 테스트 항목 | 결과 | 스크린샷 |
|--------|-------------------|------|-------|
| 시나리오 1 | 팀 생성 및 초대 코드 발급 | ✅ 성공 | 01~03 |
| | 초대 코드 기반 팀 참여 | ✅ 성공 | 04~07 |
| 시나리오 2 | 팀장의 일정 생성 | ✅ 성공 | 08 |
| | 팀원의 일정 조회 권한 | ✅ 성공 | 09 |
| 시나리오 3 | 일정 충돌 감지 (409 예러) | ✅ 성공 | 10 |
| | 충돌 없는 일정 생성 | ✅ 성공 | 11 |
| | 팀원의 변경 요청 생성 | ✅ 성공 | 12~14 |
| | 팀장의 변경 요청 승인 | ✅ 성공 | 15~16 |
| 시나리오 4 | 팀원 권한 예외 처리 | ✅ 성공 | 17 |

총 테스트 케이스: 10개 성공: 10개 (100%) 실패: 0개 스크린샷: 17개

5. 통합 테스트

❖통합 테스트 결과 예시 3

시나리오별 상세 결과

시나리오 1: 신규 팀 생성 및 초대 코드를 통한 팀원 참여

테스트 목적: 팀 생성, 초대 코드 발급, 팀원 참여 프로세스 검증

실행 내용:

1.  김개발(팀장) 회원가입 및 로그인
2.  팀 "백엔드 개발팀" 생성 → 초대 코드 **F9KFLS** 발급
3.  이코더(팀원) 회원가입 및 로그인
4.  `/teams/join` 페이지에서 초대 코드 F9KFLS 입력
5.  팀 참여 성공 → `team_members` 테이블 등록 확인
6.  캘린더 접근 권한 부여 (403 에러 없음)

검증 결과:

-  팀 ID: 61, 초대 코드: F9KFLS
-  `team_members` 테이블: `user_id` 120(팀장), 121(팀원) 등록
-  이전 테스트 문제(403 Forbidden) 완전 해결

스크린샷:

- `team-workflow-01-login-success.png`: 김개발 로그인
- `team-workflow-02-team-created.png`: 팀 생성 완료
- `team-workflow-03-invite-code.png`: 초대 코드 F9KFLS
- `team-workflow-04-leecoder-login.png`: 이코더 로그인
- `team-workflow-05-join-page.png`: 팀 참여 페이지
- `team-workflow-06-join-success.png`: 팀 참여 성공
- `team-workflow-07-calendar-access-success.png`: 캘린더 접근 성공

6. 개발된 팀캘린더톡 앱

❖화면 예시 1 : 팀 캘린더

팀캘린더톡

대시보드 팀 캘린더 **MyApp 개발 팀1** 원형설님 로그아웃

MyApp 개발 팀1 

새로고침 + 새 일정

오늘 이전 다음

2025년 10월

월 주 일

| 일 | 월 | 화 | 수 | 목 | 금 | 토 |
|-----------|----|----|----|----|----|----|
| 28 | 29 | 30 | 01 | 02 | 03 | 04 |
| 05 | 06 | 07 | 08 | 09 | 10 | 11 |
| API 서버 점검 | | | | | | |
| 12 | 13 | 14 | 15 | 16 | 17 | 18 |
| 코드 리뷰 세션 | | | | | | |
| 19 | 20 | 21 | 22 | 23 | 24 | 25 |
| 26 | 27 | 28 | 29 | 30 | 31 | 01 |

팀 채팅 

10월 5일 (일) 

아직 메시지가 없습니다

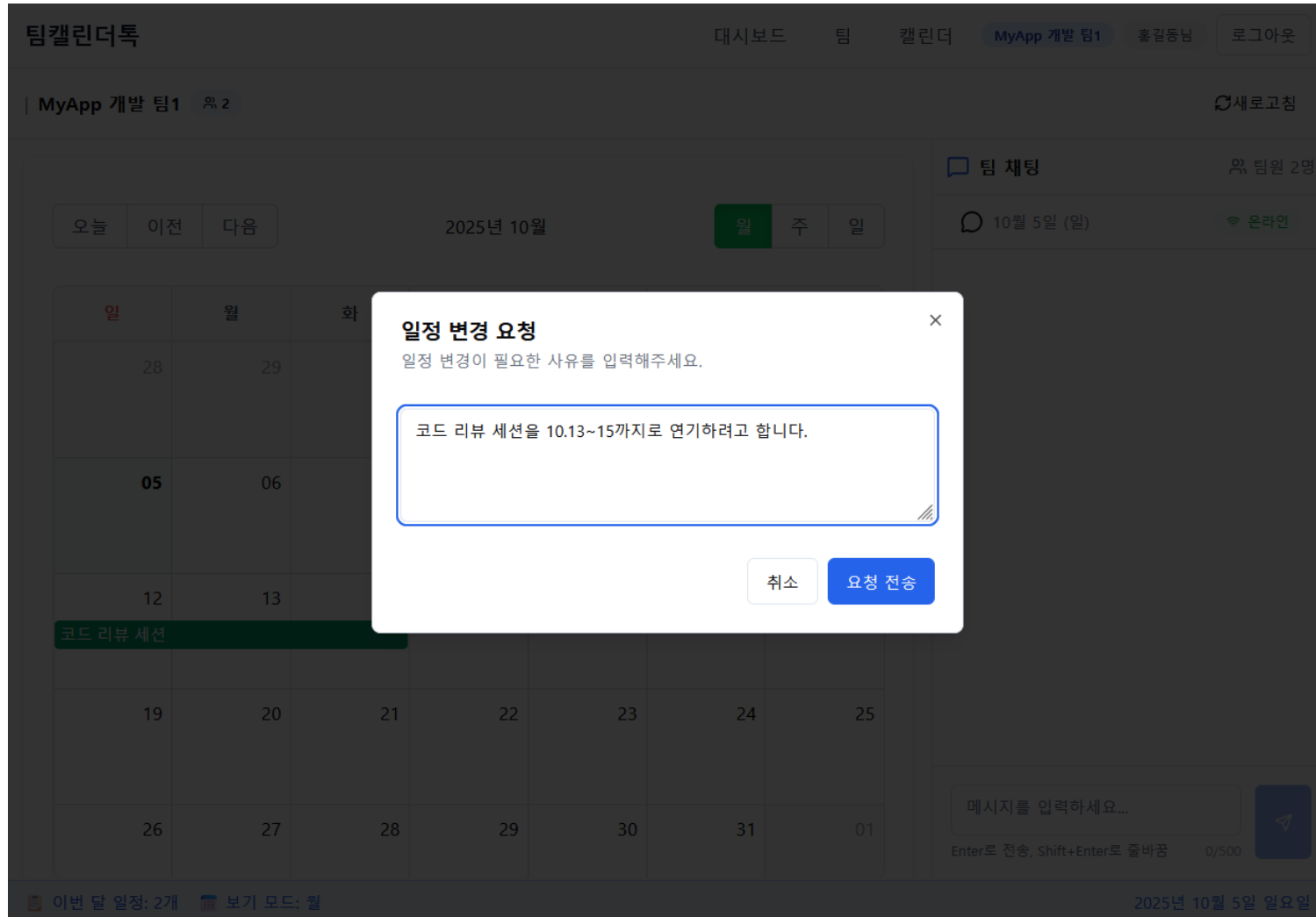
첫 번째 메시지를 보내보세요!

메시지를 입력하세요...

Enter로 전송, Shift+Enter로 줄바꿈 0/500

6. 개발된 팀캘린더톡 앱

❖화면 예시 2 : 일정 변경 요청 화면



6. 개발된 팀캘린더톡 앱

❖화면 예시 3 : 팀장의 캘린더 속 승인화면

팀캘린더톡

대시보드 팀 캘린더 **MyApp 개발 팀1** 원형성님 로그아웃

MyApp 개발 팀1 2

새로고침 + 새 일정

오늘 이전 다음

2025년 10월

월 주 일

| 일 | 월 | 화 | 수 | 목 | 금 | 토 |
|----------|----|----|-----------|----|----|----|
| 28 | 29 | 30 | 01 | 02 | 03 | 04 |
| 05 | 06 | 07 | 08 | 09 | 10 | 11 |
| | | | API 서버 점검 | | | |
| 12 | 13 | 14 | 15 | 16 | 17 | 18 |
| 코드 리뷰 세션 | | | | | | |
| 19 | 20 | 21 | 22 | 23 | 24 | 25 |
| 26 | 27 | 28 | 29 | 30 | 31 | 01 |

팀 채팅 2명

10월 5일 (일) 온라인

대기 중인 요청 1건

! 일정 변경 요청 요청 10월 5일 오후 04:09

📅 코드 리뷰 세션

👤 홍길동

코드 리뷰 세션을 10.13~15까지로 연기하려고 합니다.

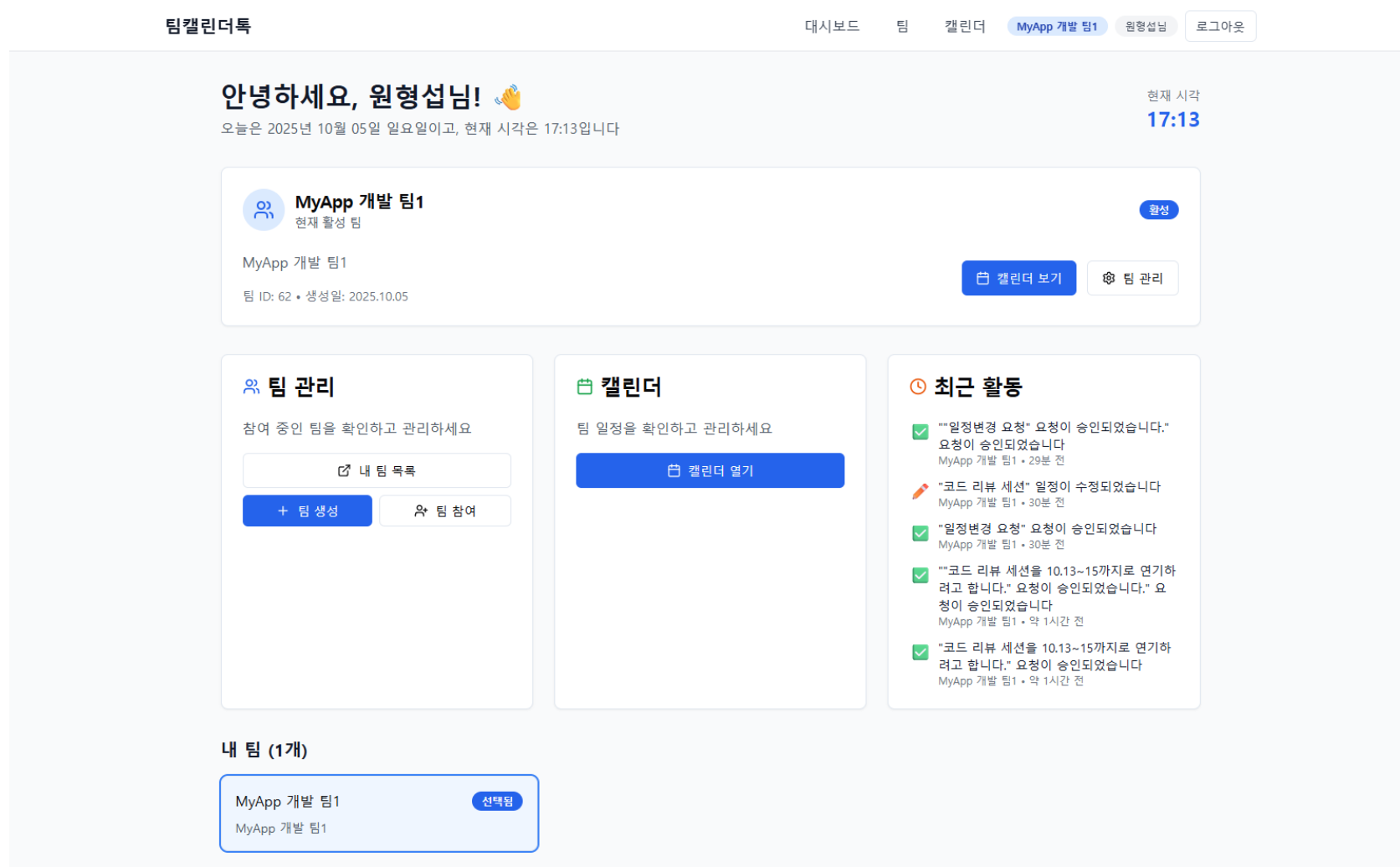
✓ 승인 ✕ 거절

메시지를 입력하세요...

Enter로 전송, Shift+Enter로 줄바꿈 0/500

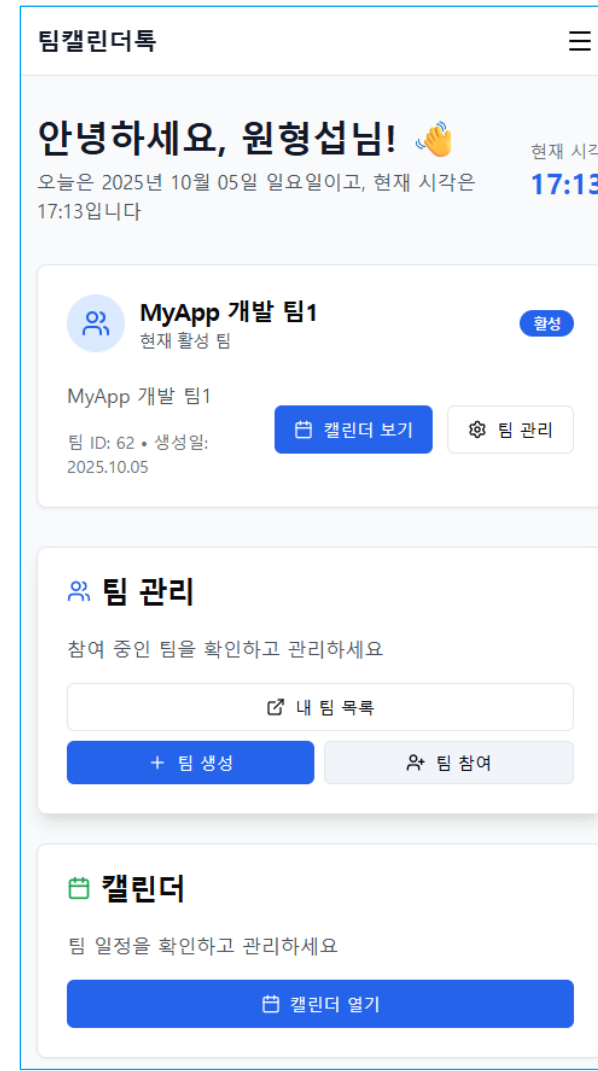
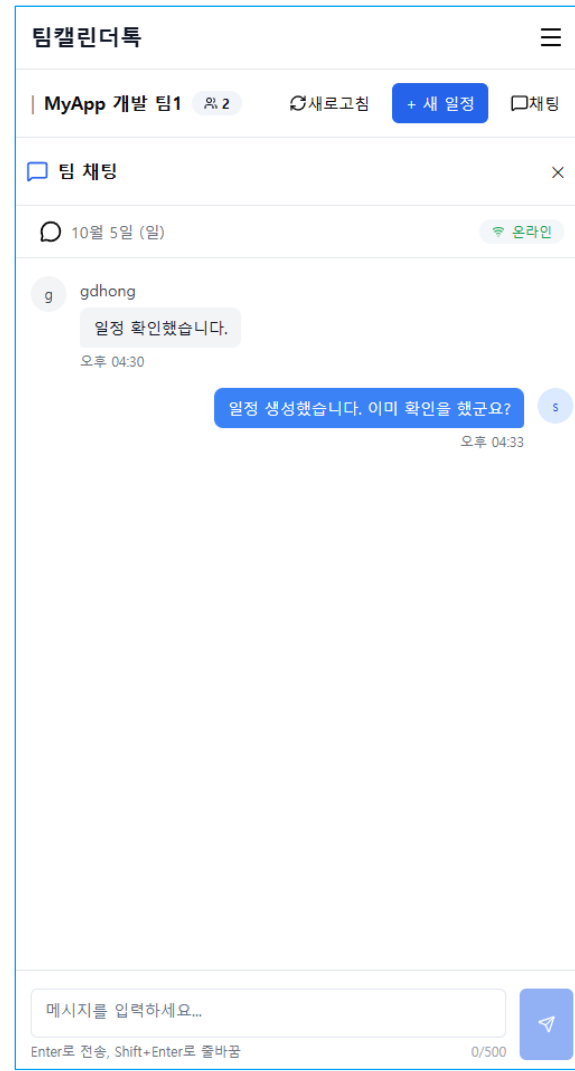
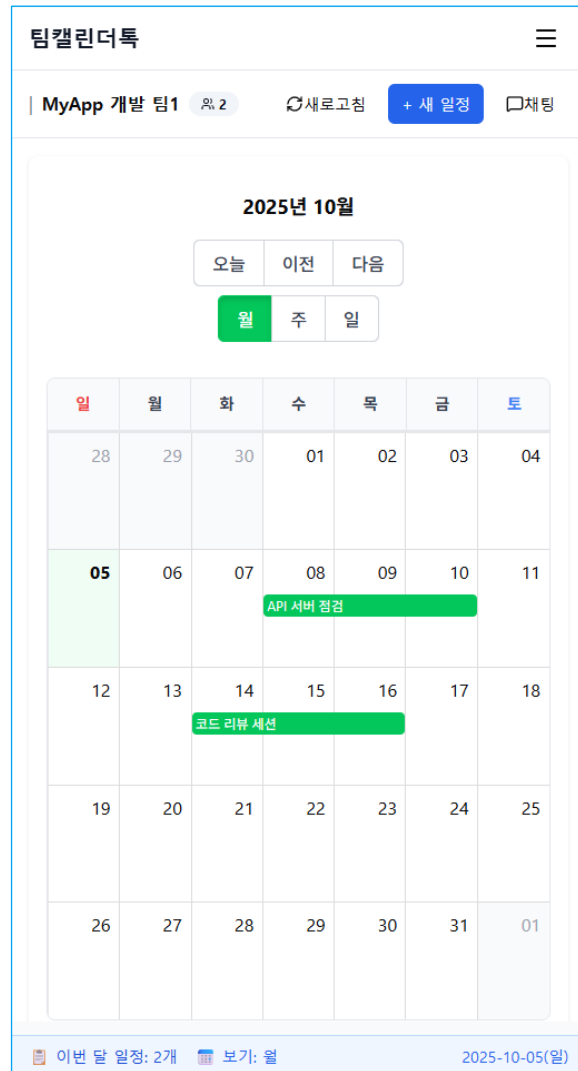
6. 개발된 팀캘린더톡 앱

❖화면 예시 4 : 대시보드 화면



6. 개발된 팀캘린더톡 앱

❖화면 예시 4 : 모바일 대응 화면



7. 정리

❖ 스타일 가이드 생성

- 참조할만한 웹사이트 화면 캡처, 와이어프레임 기반으로 스타일 가이드 생성

❖ 사전 준비

- 프론트 전용 사용자 정의 Skill 준비
- 깃헙 이슈 템플릿(제목/라벨/완료조건 등)
- 하위 CLAUDE.md 지침
- 프론트엔드 개발 전용 MCP 설정

❖ 개발 흐름

- 이슈 번호 기반 feature 브랜치 분기 → 코드, 문서 분석 → 구현 → 테스트 → 스테이징-커밋-푸시 → PR 생성/병합

❖ 주의/디버깅

- 백엔드 API는 환경변수로 주입, 선택한 기술스택을 문서화, 변경 시 Swagger/ERD 동기화, 로깅 중심 디버깅과 chrome-devtools MCP 활용

❖ 통합 테스트

- Playwright MCP로 사용자 시나리오 E2E 수행 후 리포트 생성
- 실패 시 수동 테스트 보완; 최종 산출물로 팀 캘린더톡 화면 예시 제공